

§ OPEN-SOURCE TOOL • SELF-HOSTER GUIDE

Status Report Autopilot

An open-source weekly-cadence approval queue for digital agencies. This guide walks through running it on your own infrastructure: install, first-run, AI model setup, every connector, and local-model hardware sizing.

READING TIME

15 minutes

AUDIENCE

Developers & Operators

PUBLISHED

May 2026

What this is

Status Report Autopilot is a weekly-cadence approval queue for the PM at a digital agency. It reads the work that already happened in Asana, Linear, Slack, Zoom, and Microsoft Teams; writes a client-facing narrative in your agency's voice; and ships it three ways: an email to the client, a downloadable PDF, and a public read-only link. Each carries the tone, length, and sign-off you captured during onboarding, and the same paper-and-ink chrome the dashboard uses.

The tool is open source under Apache 2.0. This guide is for the operator running it on their own infrastructure. There is no hosted version. Everything below assumes you are bringing your own machine, your own AI model key, and your own connector credentials.

WHAT YOU DO NOT NEED

A Vercel / AWS / Netlify account. A registered Google or Microsoft OAuth app for sign-in. A Stripe account. Per-connector OAuth registrations on the app side. All of those were removed in the open-source repositioning. Every credential the app needs is pasted in by you, in the UI.

§ CONTENTS

01	Before You Start	02
02	Install	03
03	First Run	04
04	Choose an AI Model	05
05	Running a Local Model	07
06	Connect Your Project Tools	10
07	Onboard Your First Client	14
08	Email, Scheduler, Maintenance	15

Before You Start

What you need

- § A machine that runs **Docker** (Docker Desktop on macOS or Windows, Docker Engine on Linux).
- § About **2 GB** of free disk for the image and the Postgres volume.
- § Network access to whichever AI provider you pick, or local hardware big enough to run an open model. See Section 5.
- § An **API key** for one of Claude (Anthropic), Gemini (Google), ChatGPT (OpenAI), or OpenRouter. Alternatively, no key at all and an OpenAI-compatible endpoint on your network.

One environment variable is required

`BETTER_AUTH_SECRET` signs session cookies. It must be set before the app will boot. Generate one with:

```
openssl rand -hex 32
```

Everything else is optional; you can leave the email and cron sections blank during install and fill them in later.

Install

```
git clone <your-fork-of-status-report-autopilot>.git
cd status-report-autopilot
cp .env.example .env

# edit .env and set BETTER_AUTH_SECRET

docker compose up --build
```

What that does, in order:

- 01 Pulls `postgres:16-alpine` and starts the database with a named volume so data survives container restarts.
- 02 Builds the app image (Node 20-alpine, Next.js production build).
- 03 Waits for the database to report healthy.
- 04 Runs `drizzle-kit migrate` against the database. The step is idempotent and safe to run on every boot.
- 05 Starts the Next.js server on `http://localhost:3000`.

The first build takes a few minutes; subsequent boots are seconds.

WHERE DATA LIVES

The Postgres data lives in a Docker named volume called `postgres-data`. Your clients, reports, model config, and stored API keys all live there; nowhere else, ever. Back it up with `docker compose exec db pg_dump` when it matters.

First Run

Open `http://localhost:3000`. The middleware redirects to `/sign-in`. Four steps from there:

- 01 **Create your account.** Click *Create an account*, enter a name, email, and an 8-character-or-more password. The first account you create owns the deployment; teammates sign up the same way after you share the URL with them.
- 02 **Connect an AI model.** A signed-in user with no model configured lands on `/setup`. Pick a provider, paste your API key, click *Continue*. The picker resolves the model offline before saving, so a missing key or a typo'd model id surfaces immediately.
- 03 **Add your first client.** From the dashboard, open `/onboarding` and walk the four-step wizard: client name + email, connect at least one source, set a voice (tone + length + writing sample), set the weekly cadence. The wizard finishes by drafting the previous calendar week's report so the first thing you see is the actual workflow, not an empty queue.
- 04 **Review and approve.** The dashboard's left pane lists clients and statuses; the right pane shows the draft. Edit inline, hit *Approve & send*. The report emails the client (if Resend is configured) and the queue auto-advances to the next pending draft.

Choose an AI Model

The app is model-agnostic. The picker at `/setup` on first run, or `/settings` later, is the single place this is configured.

PROVIDER	BEST FOR	DEFAULT MODEL
Claude	Highest narrative quality. The recommended pick if reports are client-facing.	<code>claude-sonnet-4-6</code>
Gemini	Cheapest reliable option. Native JSON-schema enforcement is a plus for structured output.	<code>gemini-3-flash-preview</code>
ChatGPT	OpenAI's models.	<code>gpt-5.1</code>
OpenRouter	One key, many providers. Useful for A/B testing without managing five accounts.	<code>anthropic/claude-sonnet-4-6</code>
Local	No cloud calls. See Section 5.	You pick.

The model id is always editable. Switching providers later just changes the active row in the `settings` table; nothing else moves.

Getting an API key

Claude (Anthropic).

- 01 Create an account at `console.anthropic.com`.
- 02 Add a payment method under **Settings** → **Billing** and load at least \$5 of credit. Anthropic keys do not produce useful output until the account has prepaid credit.
- 03 Open **Settings** → **API Keys** → **Create Key**. Name the key (e.g. `status-report-autopilot`), copy the value. It starts with `sk-ant-`.

Gemini (Google).

- 01 Visit `aistudio.google.com/apikey`. A Google account is required.
- 02 Click **Create API key** and either create a new Google Cloud project or pick an existing one.
- 03 Copy the value. It starts with `AIza`. The free tier is generous; you do not need to enable billing for low-volume use, though the paid tier raises the rate limits.

ChatGPT (OpenAI).

- 01 Sign in at `platform.openai.com`.
- 02 Add a prepaid balance under **Settings → Billing**. OpenAI requires at least \$5 of credit before new API keys produce output.
- 03 Open the dashboard's left rail → **API keys → Create new secret key**. Name the key, leave the default permissions, copy the value. It starts with `sk-proj-` or `sk-`. You can only see the full value once; paste it into the picker immediately or save it somewhere you can copy from.

OpenRouter.

- 01 Sign in at `openrouter.ai`. Google or GitHub sign-in both work.
- 02 Add credits under **Settings → Credits**. \$5 is enough to test a few reports across multiple providers.
- 03 Open **Settings → Keys → Create Key**. Name it, optionally set a credit limit on the key itself, copy the value. It starts with `sk-or-v1-`.
- 04 OpenRouter charges a small markup over each underlying provider's cost in exchange for one account that reaches every model on the market. Useful if you want to A/B test Claude against Gemini against an open model without managing three separate billing relationships.

Local model. No key needed for an open endpoint that does not authenticate (Ollama, LM Studio defaults). See Section 5.

WHERE THE KEY LIVES

The API key is stored in the app's own Postgres database (the same database the dashboard runs on). It is never sent anywhere except the provider you chose. It does not pass through any Simkins & Elgazar service; there is none.

Running a Local Model

If you'd rather not send any client data to a hosted LLM provider, point the picker at an OpenAI-compatible endpoint on your own hardware.

Backends

BACKEND	BEST ON	WHY
Ollama	macOS / Linux / Windows	Simplest install. <code>ollama serve</code> runs on <code>localhost:11434/v1</code> out of the box, OpenAI-compatible.
LM Studio	macOS / Windows	GUI for downloading and serving models, OpenAI-compatible server.
mlx-lm / mlx-omni-server	Apple Silicon (M1+)	Fastest path on Apple hardware. Runs MLX-quantized models with hardware acceleration.
vLLM	NVIDIA GPU servers	Production-grade serving, batched inference, OpenAI-compatible.

Hardware tiers

These recommendations are calibrated for *this specific workload*: turning roughly 20 to 100 activity events per client per week into a 250 to 400-word client-facing report. Reasoning effort is moderate; the model has to follow a JSON schema reliably and not hallucinate event ids.

TIER	MEMORY BUDGET	EXAMPLE HARDWARE	EXAMPLE MODELS
Small	6–10 GB	M1 Air 16 GB, RTX 3060 12 GB	Qwen 2.5 7B, Llama 3.1 8B, Phi 3 medium
Medium RECOMMENDED	18–32 GB	M2/M3 Pro 32 GB, M2 Max 32 GB+, RTX 4090 24 GB	Qwen 2.5 32B (4-bit MLX), Llama 3.3 70B (heavily quantized)
Large	40–80 GB	M3 Ultra 192 GB, dual RTX 6000 Ada, single H100 80 GB	DeepSeek V2.5, Llama 3.1 70B (full precision), Mixtral 8×22B

STRUCTURED-OUTPUT GOTCHA

Most local backends do not enforce JSON schemas during decoding. The narrative agent compensates by injecting the schema into the prompt and parsing the model's text, which places the burden on the model to follow instructions exactly. Two practical knobs: pick a model trained for instruction following and JSON output (Qwen 2.5 32B is the most reliable we have measured for this task); and if your backend supports it, enable JSON-mode or grammar-constrained decoding.

Pointing the app at your endpoint

In the picker, choose **Local model** and fill:

- § **Endpoint URL.** For example, `http://localhost:11434/v1` (Ollama default) or `http://<your-host>:1234/v1` (LM Studio).
- § **API key.** Leave blank if your backend does not authenticate. Otherwise paste the key.
- § **Model.** The id the endpoint serves, e.g. `qwen2.5:32b-instruct-q4_K_M` for Ollama or `mlx-community/Qwen2.5-32B-Instruct-4bit` for MLX.

If the app and the model are on different machines, the endpoint URL must be reachable from the **app container**. On Docker Desktop that usually means using the machine's LAN IP, not `localhost`.

Connect Your Project Tools

Each connector uses bring-your-own-credentials. You paste a token (or, for Zoom and Microsoft Teams, an OAuth app's credential triple) into the onboarding wizard. There are no per-deployer OAuth apps to register on the app side.

You can pick more than one target per source — multiple Asana projects, multiple Slack channels, multiple Zoom hosts, and so on. The picker is a checkbox list; the report blends activity across every selected target.

Least privilege. Status Report Autopilot only ever reads from your project tools, it never writes. Where a source supports it, scope the credentials to read-only. Per-source notes are inlined below.

WHO DOES WHAT

Status Report Autopilot is self-hosted. A few different people are involved in setup, and it matters for the connectors:

- § **Deployer** — you, the agency owner or tech lead. Self-hosts the app (`docker compose up`), configures the AI model, sets `APP_URL` . One-time.
- § **Agency PM** — an account manager at the agency. Uses the dashboard, runs onboarding once per client, approves and sends the weekly report.
- § **Client (or client's IT)** — your client's contact. Provides credentials for the client's project tools where admin access is required.

Asana and Linear use **personal tokens**, so an agency PM can usually create them in the client's account on the client's behalf (with permission). Quick.

Slack, Zoom, and Microsoft Teams each require building or registering an app inside the client's workspace, which needs the client's IT or workspace admin to participate. This is a vendor limitation, not something the connector can shortcut. Plan a 15-30 minute call with the client's admin to walk through those three together; the steps below are written assuming an admin is in front of the screen.

Asana

Who: the client, or an agency PM with member access to the client's Asana account. No admin rights needed.

- 01 In Asana, open **My Settings → Apps → Manage developer apps**.
- 02 Click **Create new token**.
- 03 Copy the token (it starts with `0/`).
- 04 In the wizard's Asana row, paste it. Pick the workspace, then check the projects to track.

Asana Personal Access Tokens are full-account credentials, Asana does not let you scope a PAT to read-only. The connector only reads, so the practical hygiene is to (a) create the token specifically for this app, (b) name it so you can recognize it (e.g. `status-report-autopilot`), and (c) revoke it from the same Manage developer apps page when you stop using it. Need real scoping? Use an OAuth app + the OAuth `tasks:read` scope.

Linear

Who: the client, or an agency PM with member access to the client's Linear workspace. No admin rights needed.

- 01 Open **Settings** → **Security and access** in Linear.
- 02 Under **Personal API keys**, click **Create key**.
- 03 **Scope the key to read-only.** Linear lets you tick exactly which actions a Personal API key can perform, only check the read scopes (e.g. `read:issues`, `read:projects`, `read:teams`). Do not check any `write:` or `admin:` scopes, the connector never writes.
- 04 Copy the key (it starts with `lin_api_`).
- 05 Paste in the wizard's Linear row. Pick the team, then check the projects to track.

Slack

Who: the client's Slack **workspace admin**. Slack requires permission to create and install apps in the workspace, and only an admin can grant the consent in step 3. Block off ~15 minutes with the admin.

Slack requires a small custom app. There is no shorter path that lets the connector read a channel.

- 01 Go to `api.slack.com/apps` and click **Create New App** → **From scratch**. Name it anything.
- 02 Under **OAuth & Permissions** → **Scopes** → **Bot Token Scopes**, add three scopes:
 - 03 `channels:read`
 - 04 `channels:history`
 - 05 `users:read`
- 06 Click **Install to Workspace** and approve.
- 07 Copy the **Bot User OAuth Token** (it starts with `xoxb-`).
- 08 Invite the bot to each channel you want to track (`/invite @your-bot-name` from inside the channel itself). The bot only sees channels it has been invited to.
- 09 Paste the token in the wizard's Slack row. Check the channels to track.

All three scopes above are read-only. **Do not add any** `:write` **or** `:manage` **scopes**, the connector never writes to Slack.

Zoom

Who: the client's Zoom **account owner or admin**. Server-to-Server OAuth apps can only be built by accounts with the right Marketplace privileges. Block off ~20 minutes with the admin.

The connector reads cloud-recording transcripts via the Server-to-Server OAuth grant. One Zoom app per Zoom account; the same credentials cover every Zoom user you track from the picker.

- 01 Go to `marketplace.zoom.us` → **Build App** → **Server-to-Server OAuth**.
- 02 Fill basic information. Activate the app.
- 03 Under **Scopes**, add:
 - 04 `user:read` (or `user:read:admin`)
 - 05 `cloud_recording:read` (or `cloud_recording:read:admin`)
- 06 Note the **Account ID**, **Client ID**, and **Client Secret** from the app's credentials page.
- 07 Paste all three in the wizard's Zoom row. Check the users whose recordings this client's reports draw from.

Both scopes above are read-only. **Do not add any** `:write` **or** `:update` **scopes**, the connector never writes to Zoom.

When a recording has a transcript file, the connector pulls the VTT and rides a plain-text excerpt along as the event detail. That is the reason Zoom earns a connector over plain calendar data.

Microsoft Teams

Who: the client's **Microsoft Entra ID (Azure AD) tenant admin, or someone with the Application Administrator role**. Registering an app, creating a secret, and granting admin consent to the Graph permissions in step 6 each need elevated rights. This one usually means looping in the client's IT department. Block off ~30 minutes.

The Teams connector reads channel messages via Microsoft Graph using app-only (client-credentials) auth.

- 01 Sign in to `entra.microsoft.com` and open **Applications** → **App registrations** → **New registration**.
- 02 Name the app (e.g. *Status Report Autopilot*), leave the redirect URI blank, click **Register**.
- 03 Note the **Application (client) ID** and the **Directory (tenant) ID** from the overview page.
- 04 Open **Certificates & secrets** → **New client secret**. Copy the **Value**; you cannot see it again.
- 05 Open **API permissions** → **Add a permission** → **Microsoft Graph** → **Application permissions** and add:
 - 06 `Team.ReadBasic.All`
 - 07 `Channel.ReadBasic.All`
 - 08 `ChannelMessage.Read.All`

09 Click **Grant admin consent** for your tenant.

10 Paste the tenant id, client id, and secret value in the wizard's Microsoft Teams row. Pick a team, then check the channels to track. Channels from a single connection must belong to the same team; add a second Teams connection to track another team's channels.

All three Graph permissions above end in `.Read` or `.ReadBasic`, they are read-only. **Do not add any `.ReadWrite` or `.Write` permissions**, the connector never writes to Teams or your tenant.

Onboard Your First Client

With credentials in hand, sign in to the dashboard, click **Set up your first client**, and walk the four-step wizard:

- 01 **Client identity.** The client's display name (shown in the dashboard) and the email an approved report sends to.
- 02 **Connect sources.** For each of the five connectors: paste credentials, pick the workspace, check the projects / channels / hosts to track. You can pick more than one per source; the report blends activity across every selected target.
- 03 **Voice.** Tone (buttoned / professional / warm), length (headlines / balanced / thorough), the sign-off line, and an optional writing sample. The narrative agent mirrors the sample's rhythm and word choice, not its content.
- 04 **Cadence.** Weekday + time-of-day + timezone for the recurring weekly report. The first draft is generated immediately; the recurring report fires on this slot every week thereafter.

After step 4, the wizard runs the **magic moment**, it pulls every connected source, runs the narrative agent against your configured model, and lands on a first-draft preview. Hit the primary button to save the client and continue to the dashboard.

The honest-window gotcha

The magic moment looks at the **previous Monday-Friday calendar week**, the same window the weekly recurring report covers. If the connected tools had no real activity in that window, the wizard shows an *"insufficient activity"* screen and **does not** save a half-onboarded client. The report agent never pads a thin week.

In practice this bites in two cases:

- § **Onboarding a brand-new client whose tools have only this-week activity.** The previous Mon-Fri is empty, so the magic moment is insufficient. Wait until the next Monday to onboard, or pick a moment when the previous week had real work.
- § **Onboarding mid-week against a client whose prior week was genuinely quiet** (a holiday, a launch retrospective, time off). Same outcome.

Recovery options when you hit insufficient: close the wizard and onboard the client again once a meaningful Monday-Friday has passed, or onboard a more active client first.

The recurring report path doesn't have this constraint, once a client is on the roster, every week's cadence slot fires its own attempt; insufficient weeks land on the dashboard as honestly-labeled insufficient reports (not lost).

Email, Scheduler, Maintenance

Email delivery

Approval-and-send emails go through Resend.

- 01 Create a Resend account, verify a domain you control.
- 02 Generate an API key.
- 03 In `.env`:

```
RESEND_API_KEY=re_...
EMAIL_FROM="Your Agency <reports@yourdomain.com>"
APP_URL=https://reports.yourdomain.com
```

Restart with `docker compose up -d`.

WITHOUT RESEND

The app still works. Approving a report marks it sent and skips the email step; the shareable `/r/[token]` link and the PDF download keep working. Add Resend whenever you are ready to push reports over email.

Recurring generation

The app runs its own hourly scheduler. Each tick reads the roster, finds clients whose cadence slot has arrived this week and have no report yet, pulls the prior Monday–Friday week's activity from every connected source, runs the narrative agent, and writes a draft into the dashboard queue.

You do not have to enable this; it runs whenever the container is up. The `/api/cron/generate-reports` route is also available for ad-hoc force-runs:

```
# Set CRON_SECRET in .env, then:
curl -H "Authorization: Bearer $CRON_SECRET" \
  https://your-app/api/cron/generate-reports
```

Without `CRON_SECRET` the route refuses to run, so generation is never publicly triggerable.

Updating

```
git pull
docker compose up --build
```

The entrypoint applies new migrations on boot, so an upgrade is one step.

Backups

```
docker compose exec db pg_dump -U postgres status_report_autopilot > backup.sql
```

Restore with:

```
docker compose exec -T db psql -U postgres status_report_autopilot < backup.sql
```

`docker compose logs -f app` follows the app's logs, including the hourly scheduler's `[scheduler] run complete` summaries.

Troubleshooting

SYMPTOM	WHAT IT USUALLY MEANS
<code>BETTER_AUTH_SECRET</code> is required at boot	The variable isn't set in <code>.env</code> . Generate one with <code>openssl rand -hex 32</code> and put it in <code>.env</code> before <code>docker compose up</code> .
App is up but <code>/</code> always redirects to <code>/setup</code>	The <code>settings</code> table does not have a model row, or the env-fallback model is unreachable. Save a model on <code>/setup</code> .
"That access token was rejected" on a connector	The pasted token is wrong, expired, or does not carry the required scopes. Re-check Slack's three scopes especially.
Zoom returns no meetings even though there are recordings	The Server-to-Server app needs <code>cloud_recording:read</code> admin scope to see another user's recordings, or you picked a Zoom user with no recordings in the window.
Microsoft Graph rejects the credentials	Admin consent was not granted, or the secret value is wrong. Entra shows the secret value once at creation time; you may have copied the secret <code>id</code> instead.
Malformed JSON from the narrative agent (local model only)	The local model is below the size or quality threshold for reliable structured output. See Section 5, the structured-output gotcha.
Dashboard shows old data after a scheduler tick	The scheduler is hourly. Force a run with the <code>CRON_SECRET</code> route, or restart the container.
For anything not in this list, <code>docker compose logs -f app</code> and <code>docker compose logs -f db</code> are the right places to look.	

STATUS REPORT AUTOPILOT IS RELEASED BY THE SIMKINS & ELGAZAR AI PRACTICE
UNDER THE APACHE 2.0 LICENSE.